Modelo e eficiência computacional

Problema Computacional e Algoritmos

ICMC USP SÃO CARLOS

Sumário

- Problema Computacional
- Algoritmos

Referências

- Aula baseado no capítulo “Modelos de Computação e Eficiência de Algoritmo” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.14. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- Aula baseada no capítulo 1 de CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.3. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

Problema Computacional

- Um problema computacional é definido por uma coleção de instâncias desse problema para as quais está associada uma solução.
- Instância é um conjunto de dados particular ou específico de um problema.

Problema Computacional

- A instância de um problema consiste da entrada necessária para calcular a solução do problema.
- A quantidade de dados que descrevem uma instância definem o tamanho dessa instância.
- Esse tamanho está relacionado ao espaço de memória demandado para estabelecer os parâmetros que compõem a instância.

Problema Computacional

- Máximo Divisor Comum
  - Input: (a,b), inteiros positivos a e b
  - Output: c=MDC(a,b) maior divisor comum
- Instância do problema: (610,183)
- Tamanho: 2 valores inteiros
- Solução: 61

Problema Computacional

- Problema de ordenação:
  - Input: Uma sequência de n números $\{a_1\ a_2, a_3, \ldots, a_n\}$
  - Output: Uma reordenação da sequência em $\{a'_1, a'_2, a'_3, \ldots, a'_n\}$ tal que

    $a'_1 \leq a'_2 \leq a'_3 \leq \ldots \leq a'_n$
- Instância do problema: {8, 3, 1, 9, 5, 4}
- Tamanho: 5 valores inteiros
- Solução: {1, 3, 4, 5, 8, 9}

Algoritmo

- Conjunto finito de passos para execução de instruções, bem definido (sem ambigüidades), retornando a solução de um problema em tempo finito.
- Um algoritmo soluciona um problema computacional ao receber uma instância qualquer e...
  - Retornar uma solução para essa instância.
  - Retornar que tal instância não tem solução.

Algoritmo

- Um algoritmo deve retornar uma solução correta e eficiente para o problema computacional.
- Corretude: retornar uma saída correta para cada instância (entrada).
- Eficiência: utilizar de forma eficiente os recursos computacionais disponíveis (tempo e memória).

Algoritmo

- Corretude: vamos considerar apenas algoritmos corretos.

- Eficiência: Vamos analisar as complexidades de tempo e de espaço de um algoritmo

Algoritmo

- Complexidade de tempo: avaliação do tempo gasto pelo algoritmo na execução das instruções considerando o tamanho da instância.

- Complexidade de espaço: avaliação da quantidade de espaço em memória gasto pelo algoritmo considerando o tamanho da instância.

Algoritmo

- Exemplo: Soma de Vetores

```
1   def soma_vetor(A,B,n):
2       for i in range(0,n):
3           C[i]=A[i]+B[i]
4       return C
```

- O tempo das operações na linha 3 são constantes
  - Não crescem com o aumento de n.
- As operações na linha 3 são executadas n vezes.
  - Crescimento linear do tempo computacional total em função do tamanho n da entrada.

Algoritmo

- Exemplo: Soma de Vetores

```
1  def soma_vetor(A,B,n):
2      for i in range(0,n):
3          C[i]=A[i]+B[i]
4      return C
```

- A complexidade de espaço avalia a quantidade de memória extra.
  - Não considera a entrada ou saída do algoritmo.
- Não há uso de memória extra
  - A complexidade de espaço é constante.

Algoritmo

- Exemplo: Função Merge do Algoritmo Merge Sort

  - Função merge é chamada diversas vezes pelo algoritmo Merge Sort.
  - A função merge combina dois subvetores já ordenados em um vetor ordenado.

Pseudocódigo

```
MERGE[A, esq, meio, dir]
   V[dir - esq + 1] = []
   i = esq
   j = meio + 1
   k = 1
   enquanto i <= meio e j <= dir
      se A[i] < A[j] então
         V[k] = A[i]
         i = i + 1
      senão
         V[k] = A[j]
         j = j + 1
      k = k + 1
   enquanto i <= meio
      V[k] = A[i]
      i = i + 1
      k = k + 1
   enquanto j <= dir
      V[k] = A[j]
      j = j + 1
      k = k + 1
   para i = 1 até k
      A[esq + i] = V[i]
```

Figura adaptada de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021.

Algoritmo

- Exemplo: Função Merge do Algoritmo Merge Sort
  - Os laços não estão aninhado.
  - Os laços serão executados n (tamanho da entrada) vezes no pior caso.
  - A complexidade de tempo da função Merge é linear em função de n.

Pseudocódigo

```
MERGE[A, esq, meio, dir]
   V[dir - esq + 1] = []
   i = esq
   j = meio + 1
   k = 1
   enquanto i <= meio e j <= dir
      se A[i] < A[j] então
         V[k] = A[i]
         i = i + 1
      senão
         V[k] = A[j]
         j = j + 1
      k = k + 1
   enquanto i <= meio
      V[k] = A[i]
      i = i + 1
      k = k + 1
   enquanto j <= dir
      V[k] = A[j]
      j = j + 1
      k = k + 1
   para i = 1 até k
      A[esq + i] = V[i]
```

Figura adaptada de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021.

Algoritmo

- Exemplo: Função Merge do Algoritmo Merge Sort

  - Aloca-se V[dir – esq + 1] posições de memória para armazenar o vetor ordenado.
  - Na última chamada da função, N posições serão alocadas.
  - A complexidade de espaço é N.

Pseudocódigo

```
MERGE[A, esq, meio, dir]
   V[dir – esq + 1] = []
   i = esq
   j = meio + 1
   k = 1
   enquanto i <= meio e j <= dir
      se A[i] < A[j] então
         V[k] = A[i]
         i = i + 1
      senão
         V[k] = A[j]
         j = j + 1
      k = k + 1
   enquanto i <= meio
      V[k] = A[i]
      i = i + 1
      k = k + 1
   enquanto j <= dir
      V[k] = A[j]
      j = j + 1
      k = k + 1
   para i = 1 até k
      A[esq + i] = V[i]
```

Figura adaptada de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021.



Modelo e eficiência computacional

Eficiência Computacional e Análise de Algoritmos

Sumário

- Eficiência Computacional
- Análise de Algoritmos

Referências

- Aula baseado no capítulo “Modelos de Computação e Eficiência de Algoritmo” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.14. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- Aula baseada no capítulo 1 de CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.3. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

Eficiência Computacional

- Problema de ordenação:
  - Input: Uma sequência de n números $\{a_1\ a_2, a_3,...., a_n\}$
  - Output: Uma reordenação da sequência em $\{a'_1,a'_2, a'_3,...,a'_n\}$ tal que

    $a'_1 \leq a'_2 \leq a'_3 \leq ... \leq a'_n$
- Instância do problema: {8, 3, 1, 9, 5, 4}
- Tamanho: 5 valores inteiros
- Solução: {1, 3, 4, 5, 8, 9}

Eficiência Computacional

- Suponha que o computador A execute 10 bilhões de tarefas por segundos.
- Um excelente programador implementa o algoritmo X em linguagem de máquina no computador A, conseguindo performance da ordem de n^2, onde n é o tamanho da entrada (10^7 números).

Eficiência Computacional

- Suponha que o computador B execute 10 milhões de tarefas por segundos.
- Um programador com nível mediano implementa o algoritmo B usando Python. Ele consegue uma performance proporcional a nlogn, onde n é o tamanho da entrada.
- Lembre-se: o computador A é 1000 vezes mais rápido do que o computador B.

Eficiência Computacional

- Exemplo: Ordenar 10^7 números.

$$A = \frac{(10^7)^2}{10^{10}} = 10^4 \approx 2.7h$$

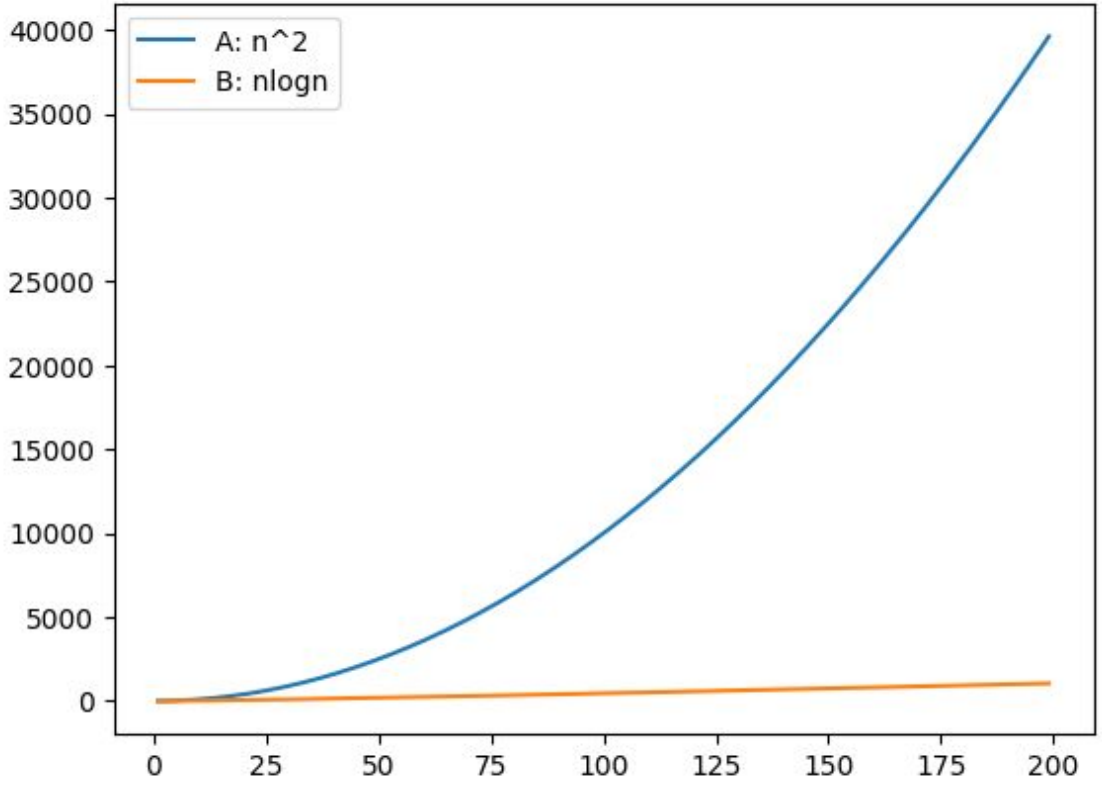
$$B = \frac{10^7 log 10^7}{10^7} = 7s$$

Eficiência Computacional

- Exemplo: Ordenar 10^7 números.

$$A = \frac{(10^7)^2}{10^{10}} = 10^4 \approx 2.7h$$

$$B = \frac{10^7 log 10^7}{10^7} = 7s$$



Eficiência Computacional

- A eficiência não recai sobre o tipo de linguagem utilizada para implementar o algoritmo, nem sobre o tipo de máquina na qual o algoritmo é executado.
- A eficiência precisa ser avaliada através de um critério adequado para comparar algoritmos.
- Esse critério deve ser independente da linguagem e tipo de máquina: complexidade de tempo e espaço

Análise de Algoritmos

- Objetivo: identificar algoritmos mais eficientes.
  - Pode existir mais de um candidato viável.
  - Descartamos algoritmos inferiores durante o processo de análise.

Análise de Algoritmos

- Podemos estabelecer os recursos necessários para execução de um algoritmo que podem ser:
  - memória;
  - largura de banda de comunicação;
  - tipo de hardware do computador.
- Todavia, a análise de algoritmos foca sobretudo na avaliação do tempo computacional.

Análise de Algoritmos

- Diferentes algoritmos que solucionam um mesmo problema podem apresentar uma diferença significativa em termos de eficiência.
- Tais diferenças podem ser mais impactantes que diferenças de hardware e software.

Análise de Algoritmos

```python
import time

def soma_v1(n):
    inicio = time.time()

    aSoma = 0
    for i in range(n):
        aSoma = aSoma + (i+1)

    fim = time.time()

    return aSoma,fim-inicio
```

```
n=10000 soma:50005000 tempo=0.003487110137939453
n=100000 soma:5000050000 tempo=0.018520832061767578
n=1000000 soma:500000500000 tempo=0.1937654018402099 6
n=10000000 soma:50000005000000 tempo=1.990614652633667
n=100000000 soma:5000000050000000 tempo=10.12375593185
Média Tempo: 2.4660287857055665
```

Análise de Algoritmos

```python
def soma_v2(n):
    inicio = time.time()

    return (n*(n+1))/2,(time.time()-inicio)
```

$$1 + 2 + 3 + \cdots + n - 1 + n = \frac{n \times (n-1)}{2}$$

```
n=10000 soma:50005000.0 tempo=1.6689300537109375e-06
n=100000 soma:5000050000.0 tempo=7.152557373046875e-07
n=1000000 soma:500000500000.0 tempo=4.76837158203125e-07
n=10000000 soma:50000005000000.0 tempo=4.76837158203125e-07
n=100000000 soma:5000000050000000.0 tempo=2.86102294921875e-06
Média Tempo: 1.239776611328125e-06
```

Análise de Algoritmos

- Segundo Knuth (apud Ziviani, 1971), a análise de algoritmos considera dois problemas:
  - Avaliar um algoritmo específico: calcular seu custo computacional, avaliando aspectos como o número de vezes que trechos do algoritmo são executados e a quantidade de memória consumida.
  - Avaliar uma classe de algoritmos: encontrar o algoritmo com melhor custo para resolver determinado problema computacional, definindo limites para o tempo de execução dos algoritmos avaliados.

1 Avaliar um algoritmo específico

Calcular seu **custo computacional**, avaliando aspectos como o **número de vezes** que trechos do algoritmo são executados e a quantidade de **memória** consumida.

Algoritmo (exemplo)

```
1  soma ← 0
2  para i← 1 até n faça
3     soma ← soma + A[i]
4  fimpara
5  se soma > 0 então
6     retorno soma
7  senão
8     retorno 0
   fimse
```

Avaliação do custo

Número de execuções
- Linha 2: n vezes
- Linha 3: n vezes
- Outras linhas: 1 vez

$\Rightarrow$ Total $\approx 2n + c$

Memória utilizada
- Vetor A: n posições
- Variáveis: constantes

$\Rightarrow$ Memória = $\Theta(n)$

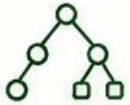
Custo computacional do algoritmo
Tempo: $\Theta(n)$
Memória: $\Theta(n)$

Objetivo: medir o custo de **UM** algoritmo específico para uma entrada de tamanho n.

2 Avaliar uma classe de algoritmos

Encontrar o algoritmo com **melhor custo** para resolver determinado problema computacional, definindo **limites** para o tempo de execução dos algoritmos avaliados.

Problema: Ordenação de n elementos

Algoritmo A (Bolha)	Algoritmo B (Inserção)	Algoritmo C (Merge Sort)	Algoritmo D (Heap Sort)
			
Tempo: $\Theta(n^2)$	Tempo: $\Theta(n^2)$	Tempo: $\Theta(n \log n)$	Tempo: $\Theta(n \log n)$
Memória: $\Theta(1)$	Memória: $\Theta(1)$	Memória: $\Theta(n)$	Memória: $\Theta(1)$



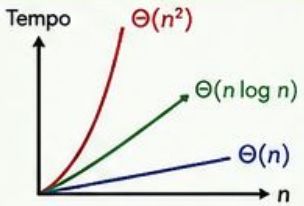
Melhor custo assintótico:
$\Theta(n \log n) \rightarrow$ **Algoritmos C ou D**



Definir limites para o tempo de execução:

- Qualquer algoritmo de comparação: $\Omega(n \log n)$ no pior caso
- Limite superior: $O(n \log n)$ (ex.: Merge Sort)

$\Rightarrow$ $\Theta(n \log n)$ é o melhor custo possível (assintoticamente) para o problema.





Objetivo: escolher, entre **UMA CLASSE DE ALGORITMOS**, aquele(s) com melhor custo para o problema.

Lembrando…Eficiência

- A eficiência não recai sobre o tipo de linguagem utilizada para implementar o algoritmo, nem sobre o tipo de máquina na qual o algoritmo é executado.
- A eficiência precisa ser avaliada através de um critério adequado para comparar algoritmos.
- Esse critério deve ser independente da linguagem e tipo de máquina: complexidade de tempo e espaço

Modelo e eficiência computacional

Avaliando Complexidade

ICMC USP SÃO CARLOS

Sumário

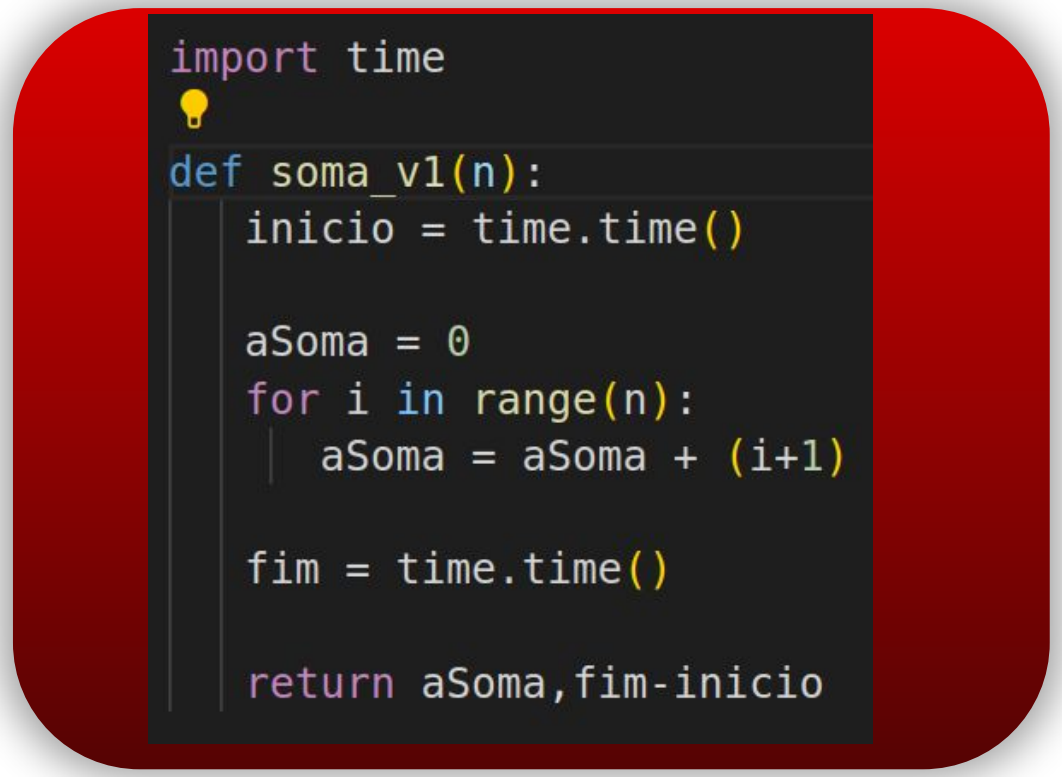
- Complexidade de Tempo
- Contagem Simplificada

Referências

- Aula baseado no capítulo “Modelos de Computação e Eficiência de Algoritmo” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.14. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- Aula baseada no capítulo 2 de CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.3. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

Complexidade de Tempo

- O tempo de execução de um algoritmo pode ser calculado considerando quanto tempo leva para executar as instruções



```python
import time

def soma_v1(n):
    inicio = time.time()

    aSoma = 0
    for i in range(n):
        aSoma = aSoma + (i+1)

    fim = time.time()

    return aSoma,fim-inicio
```

Complexidade de Tempo

- Na análise de um algoritmo, as complexidades de tempo e espaço são consideradas ao se avaliar um algoritmo.
- A complexidade de tempo se refere ao tempo gasto pelo algoritmo na execução das instruções a partir da instância recebida como entrada.
- A complexidade de espaço se refere ao espaço em memória gasto pelo algoritmo a partir do tamanho da instância recebida como entrada.

```python
def soma_v1(n):
    aSoma = 0
    for i in range(n):
        aSoma = aSoma + (i+1)

    return aSoma
```

```python
def soma_v2(n):
    return (n*(n+1))/2
```

Complexidade de Tempo

- Tempo de execução também pode variar:
  - de acordo com o algoritmo,
  - para um mesmo algoritmo implementado de diferentes formas,
  - para uma mesma implementação executada em computadores diferentes,
  - a estimativa de tempo não pode ser baseada em entradas de tamanho reduzido.

Contagem Simplificada

- Vamos definir uma forma de analisar a complexidade de um algoritmo sem considerar aspectos como modelo do processador, frequência de clock, sistema operacional e compilador.
- O objetivo é comparar o desempenho de diferentes soluções para problemas computacionais retornadas por algoritmos quando executando instâncias com tamanho grande.

Contagem Simplificada

- Contagem das operações executadas:
  - Depende do algoritmo.
  - Depende do tipo de implementação.
  - Não depende do tipo de máquina.
  - Nem sempre são claras quais operações serão contadas.

```
def soma1(n):
    soma = 0
    for i in range(n):
        soma +=i
    return soma
```

Contagem Simplificada

1op: atrib
2op: atrib + soma -> n vezes
2op: atrib + soma -> n vezes
1op: retorno

Total = 1+2n+2n+1=4n+2

```
def soma2(n):
    soma = 0
    i=0
    while i< n:
        soma +=i
        i+=1
    return soma
```

Contagem Simplificada

```
while i< n:
        soma +=i
        i+=1
n=3
```

```
0< n:
soma +=0
i+=1
```

```
1< n:
soma +=1
i+=1
```

```
2<n
soma += 2
i+=1
```

```
def soma2(n):
    soma = 0
    i=0
    while i< n:
        soma +=i
        i+=1
    return soma
```

Contagem Simplificada

1op: atrib
1op: atrib
1op: comparação -> n vezes
2op: atrib + soma -> n vezes
2op: atrib + soma -> n vezes
1op: retorno

Total = 1+1+n+2n+2n+1 = 5n+3

Contagem Simplificada

- Temos:
  - soma1(): 4n+2
  - soma2(): 5n+3
- Tanto soma1() quanto soma 2() apresentam ordem de crescimento linear no tempo de execução em função do tamanho da entrada n.

```
def pot(a,n):
    if (b == 1):
        return a
    else:
        return a*pot(a, n-1)
```

Contagem Simplificada

1op: comparação -> 1
1op: retorno -> 1
1op: comparação -> 1
2op: retorno + produto + chamada função + subtração -> 5

n=1 Total(1) = 2
n Tota(n)= 5+T(n-1)

```
def pot(a,n):
    if (b == 1):
        return a
    else:
        return a*pot(a, n-1)
```

Contagem Simplificada

n=1 Total(1) = 2

n Tota(n)=5+T(n-1)

=5+[5+T(n-2)]=2.5 + T(n-2)=

=2.5+[5+T(n-3)] = 3.5 + T(n-3)

…=k.5+T(n-k)

k=n-1 -> Total(n)=(n-1).5 + T(n-(n-1))=

=5n-5 + T(1) = 5n-5+2

Total(n)=5n-3

Contagem Simplificada

- O modelo detalhado dificulta a análise da complexidade de tempo dos algoritmos.
  - A precisão obtida não é maior do que outros modelos de contagem simples.
- O modelo de contagem simples leva em consideração apenas as operações e os termos mais custosos.

ICMC USP SÃO CARLOS



Modelo e eficiência computacional

Complexidade de Pior e Melhor Caso

Sumário

- Complexidade e Custo Computacional
- Complexidade de Pior e Melhor Caso

Referências

- Aula baseado no capítulo “Modelos de Computação e Eficiência de Algoritmo” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.14. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/

Entrada	N	N log N	N^2	N^3	2^N	N!
10	< 1 seg	< 1 seg	< 1 seg	< 1 seg	< 1 seg	4 seg
30	< 1 seg	< 1 seg	< 1 seg	< 1 seg	18 min	10^{25} anos
50	< 1 seg	< 1 seg	< 1 seg	< 1 seg	36 anos	Muito tempo
100	< 1 seg	< 1 seg	< 1 seg	1 seg	10^{17} anos	Muito tempo
1 000	< 1 seg	< 1 seg	1 seg	18 min	Muito tempo	Muito tempo
10 000	< 1 seg	< 1 seg	2 min	12 dias	Muito tempo	Muito tempo
100 000	< 1 seg	2 seg	3 horas	32 anos	Muito tempo	Muito tempo
1 000 000	1 seg	20 seg	12 dias	31 710 anos	Muito tempo	Muito tempo

Fonte: Adaptado de Kleinberg e Tardos (2006).

Complexidade e Custo Computacional

- Execução dos algoritmos em um computador com processador de frequência 1 GHz
- Nas entradas até 10.000, algoritmos com complexidades N e N log N apresentam desempenho parecido (<1 seg).
- Para entrada de 1 milhão, temos N em 1 s e N log N em 20 s.

Entrada	N	N log N	N^2	N^3	2^N	N!
10	< 1 seg	< 1 seg	< 1 seg	< 1 seg	< 1 seg	4 seg
30	< 1 seg	< 1 seg	< 1 seg	< 1 seg	18 min	10^{25} anos
50	< 1 seg	< 1 seg	< 1 seg	< 1 seg	36 anos	Muito tempo
100	< 1 seg	< 1 seg	< 1 seg	1 seg	10^{17} anos	Muito tempo
1 000	< 1 seg	< 1 seg	1 seg	18 min	Muito tempo	Muito tempo
10 000	< 1 seg	< 1 seg	2 min	12 dias	Muito tempo	Muito tempo
100 000	< 1 seg	2 seg	3 horas	32 anos	Muito tempo	Muito tempo
1 000 000	1 seg	20 seg	12 dias	31 710 anos	Muito tempo	Muito tempo

Fonte: Adaptado de Kleinberg e Tardos (2006).

Complexidade e Custo Computacional

- Algoritmos com complexidades N^2 e N^3 apresentam custos maiores.
- Para entrada = 1.000, o algoritmo com N^2 leva 1 s e N^3 gasta 18 min.

Entrada	N	N log N	N^2	N^3	2^N	N!
10	< 1 seg	< 1 seg	< 1 seg	< 1 seg	< 1 seg	4 seg
30	< 1 seg	< 1 seg	< 1 seg	< 1 seg	18 min	10^{25} anos
50	< 1 seg	< 1 seg	< 1 seg	< 1 seg	36 anos	Muito tempo
100	< 1 seg	< 1 seg	< 1 seg	1 seg	10^{17} anos	Muito tempo
1 000	< 1 seg	< 1 seg	1 seg	18 min	Muito tempo	Muito tempo
10 000	< 1 seg	< 1 seg	2 min	12 dias	Muito tempo	Muito tempo
100 000	< 1 seg	2 seg	3 horas	32 anos	Muito tempo	Muito tempo
1 000 000	1 seg	20 seg	12 dias	31 710 anos	Muito tempo	Muito tempo

Fonte: Adaptado de Kleinberg e Tardos (2006).

Complexidade e Custo Computacional

- Algoritmos 2^N e N! são menos eficientes.
- Para entrada=50, 2^N leva 36 anos e N! leva um tempo inviável.

Complexidade e Custo Computacional

- A complexidade de tempo é proporcional à quantidade de operações básicas executadas pelo algoritmo.
- Operações básicas: operação aritmética entre variáveis do algoritmo, ou uma comparação entre duas variáveis, ou uma atribuição de valor a uma variável.

Complexidade e Custo Computacional

- Nesse contexto, a eficiência de um algoritmo pode ser definida como sua complexidade de tempo.
- A complexidade de tempo é dada pelo número de instruções básicas que ele executa considerando o tamanho da entrada.

Complexidade e Custo Computacional

- Todavia, não se faz necessário contar todas as operações elementares como feito na contagem simplificada.
- Deve-se identificar onde o gasto de tempo do algoritmo é proporcional ao número de execuções de determinadas operações.

Complexidade de Pior e Melhor Caso

- Melhor caso: tempo mínimo de execução para todas as entradas possíveis de um dado tamanho n.
- Caso médio: tempo médio de execução para todas as entradas possíveis de um dado tamanho n, ou estimativa sobre o tempo de execução.
- Pior caso: tempo máximo de execução para todas as entradas possíveis de um dado tamanho n.

Complexidade de Pior e Melhor Caso

- Busca Sequencial:
  - Entrada: L (lista), e (elemento da busca)
  - Saída: True (encontrado) ou False.
  - N= len(L) - tamanho da entrada;

```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Complexidade de Pior e Melhor Caso

- Busca Sequencial:
  - Percorre uma lista do início ao fim.
  - Se valor buscado for encontrado, retorna verdadeiro.
  - Caso contrário, a execução continua até a lista acabar, retornando falso.

```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Complexidade de Pior e Melhor Caso

- Busca Sequencial
  - O melhor caso ocorre quando o elemento buscado é o primeiro da lista.
  - Não importa se o tamanho da entrada é N = 8 ou N = 1 bilhão.
  - A complexidade de tempo do melhor caso é constante (1).

```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Complexidade de Pior e Melhor Caso

- Busca Sequencial:
  - O pior caso ocorre quando o elemento buscado não está na lista ou é o último elemento.
  - Toda a lista é percorrida.
  - A complexidade de tempo do pior caso será linear em N.

```
3  def busca(L,e):
4      for i in range(len(L)):
5          if L[i] == e:
6              return True
7      return False
```

Complexidade de Pior e Melhor Caso

- Progressão Geométrica:

  $a_n = a_1 . r^{n-1}$

  - Entrada: a (termo inicial), r (razão), n (índice do termo a ser calculado).
  - Saída: valor do n-ésimo termo.

```
1  def pgIter(a,r,n):
2      '''
3      Cálculo do n-ésimo
4       termo de uma PG
5       '''
6      soma=a
7      for i in range(1,n):
8          soma = soma*r
9      return soma
```

Complexidade de Pior e Melhor Caso

- Progressão Geométrica:
  - A complexidade de tempo é linear pelo laço existente na linha 7
  - Um pior e melhor caso não são definidos já que o valor da entrada não afeta o número N de iterações executadas pelo laço.
  - O pior e melhor caso teriam a mesma complexidade.

```
1  def pgIter(a,r,n):
2      '''
3      Cálculo do n-ésimo
4       termo de uma PG
5      '''
6      soma=a
7      for i in range(1,n):
8          soma = soma*r
9      return soma
```

Complexidade de Pior e Melhor Caso

- Progressão Geométrica:
  - A solução mais eficiente, usando **forma fechada**, também tem o mesmo desempenho no melhor e pior caso de sua execução.
  - Não há laços e sim uma instrução de cálculo.
  - Logo, a complexidade de tempo de melhor e pior caso é constante.

```
1  def pgFormaFechada(a,r,n):
2      '''
3      Cálculo do n-ésimo
4      termo de uma PG
5      '''
6      return a*r**(n-1)
```

Complexidade de Pior e Melhor Caso

- Observa-se que o pior caso possível é considerado na determinação do tempo máximo para solucionar uma instância de grande porte.
- Trata-se de uma análise assintótica, ou seja, avalia-se o comportamento do algoritmo para entradas de grande porte.



Modelo e eficiência computacional

RAM Model

Referências

- Aula baseado no capítulo “Modelos de Computação e Eficiência de Algoritmo” de SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.14. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/
- Aula baseada no capítulo 2 de CORMEN, Thomas. Algoritmos - Teoria e Prática. 3rd ed. Rio de Janeiro: GEN LTC, 2012. E-book. p.3. ISBN 9788595158092. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788595158092/

ICMC USP SÃO CARLOS

RAM Model

- Uma forma de medir a complexidade de um algoritmo é através de um modelo de computação genérico.
- Random Access Machine - Máquina de Acesso Aleatório - é um modelo abstrato de máquina computacional contendo instruções normalmente executadas por computadores reais.

RAM Model

- Segundo Cormen (2024):
  - Um detalhamento excessivo tornaria o processo tedioso e reduziria a percepção da análise e do projeto de algoritmos.
  - Algo vago, também tornaria o model irreal.
    - Por exemplo, supor que o modelo execute uma ordenação com uma única instrução.

RAM Model

- Algumas hipóteses sobre o modelo serão assumidas.
  - Opera de forma sequencial:
    - as instruções são executadas uma após a outra , ou seja, não há operações concorrentes.
  - Um único processador lida com as operações.
  - Algoritmos são implementados como programas.

RAM Model

- Tipos de instruções consideradas:
  - aritméticas (adicionar, subtrair, multiplicar, dividir, resto, piso, teto),
  - transferências de dados (carregar, armazenar, copiar),
  - estruturas de controle (condicional, repetição, chamadas e retornos de sub-rotinas)
- Cada instrução consome uma quantidade de tempo constante.

RAM Model

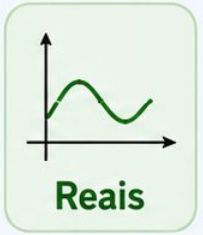
- Tipos básicos: inteiros e reais.
- Um limite no tamanho de cada palavra de dados é assumido.
  - Por exemplo, assumimos que inteiros são representados por clgn bits para alguma constante c≥1 e entradas de tamanho n.
  - c≥1 garante que a palavra contenha n
  - c≥1 sendo um valor constante impede que a palavra cresça indeterminadamente

Modelo RAM

Tipos básicos e tamanho limitado da palavra

1 Tipos básicos

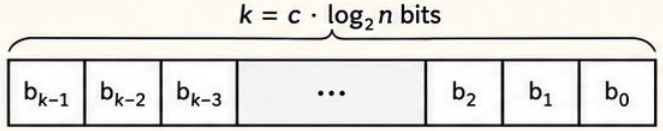




O modelo assume tipos básicos como inteiros e reais.

2 Palavra de tamanho limitado

palavra $= c \cdot \log_2 n$ **bits**

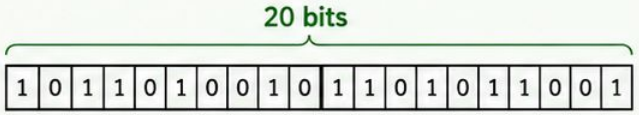


Para uma entrada de tamanho n, cada palavra tem tamanho proporcional a $\log n$.

Para um dado n, o tamanho da palavra é fixo.

3 Exemplo

- $n = 1024 \Rightarrow \log_2(n) = 10$
- Se $c = 2$, então a palavra tem 20 bits

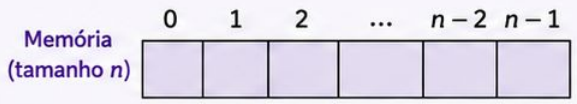




10 bits são suficientes para indexar $2^{10} = 1024$ posições de memória.

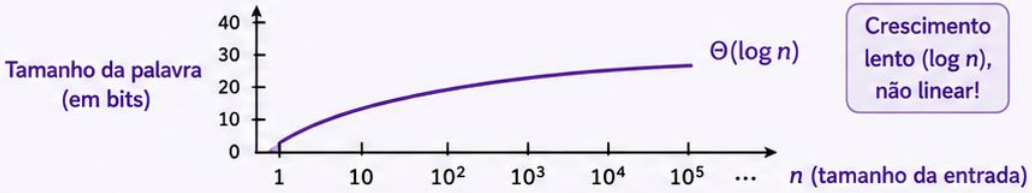
Por que $c \geq 1$?

1 $c \geq 1$ garante espaço suficiente para representar endereços de memória de tamanho n.



Precisamos de $\lceil \log_2 n \rceil$ bits para representar os endereços.

2 Como c é constante, a palavra não cresce indefinidamente; ela cresce apenas de forma logarítmica com n.





Ideia central: no modelo RAM, cada operação manipula **palavras de tamanho limitado**, tipicamente $c \cdot \log n$ bits.

RAM Model

- Uma função de complexidade de tempo T(n) é utilizada para medir o tempo de execução de um algoritmo para uma instância de tamanho n.

```
    T(n)= 4n+2

def soma1(n):
    soma = 0
    for i in range(n):
            soma +=i
    return soma
```

RAM Model

- Permite uma adequada comparação entre algoritmos diferentes aplicados a um mesmo problema.
- Facilita a avaliação do comportamento dos algoritmos em instâncias de grande porte.

RAM Model

- O modelo RAM não considera hierarquia da memória em computadores.
  - Caches ou memória virtual não são tratados.
- A hierarquia de memória pode impactar na análise de programas reais em máquinas reais.

RAM Model

- Computadores reais contêm instruções que não são consideradas no modelo RAM mencionado.
- Por exemplo, várias instruções ocorrem no cálculo x^y para x e y reais.
- Porém, x^y pode ser considerada uma operação de tempo constante sob certas restrições.

RAM Model

- Podemos assumir que temos:
  - Memória de Acesso Aleatório Infinito (uma matriz).
  - Cada célula tem um endereço único.
  - Cada célula armazena uma palavra: um inteiro, um caractere, um endereço, etc.
  - A entrada é armazenada na Memória de Acesso Aleatório .
  - A saída é escrita na Memória de Acesso Aleatório .
  - Há também um número constante de registradores.

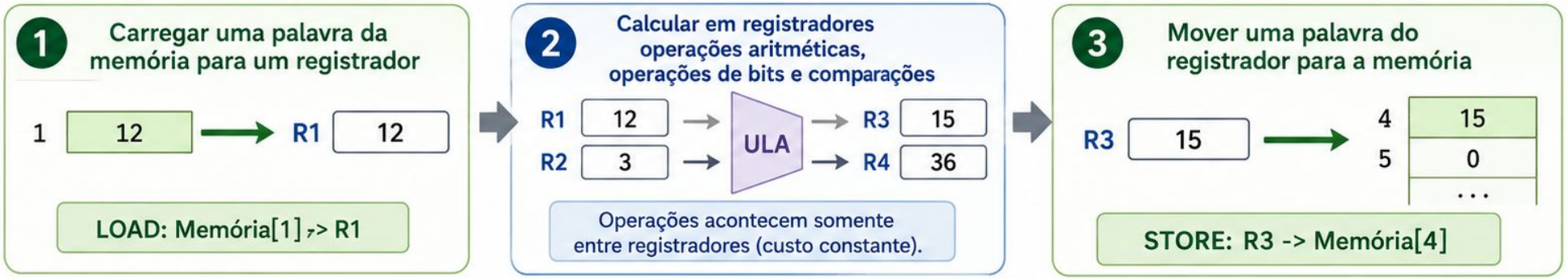
RAM Model

- A cada passo conseguimos executar as instruções de:
  - Carregar uma palavra da memória para um registrador
  - Calcular em registadores operações aritméticas (+, −, *, /), operações de bits, comparações, entre outras.
  - Mover uma palavra do registrador para a memória

Modelo RAM

Em cada passo, o modelo assume operações básicas com custo constante.





Ideia central: ler da memória, processar nos registradores e gravar de volta na memória.

RAM Model

- Um algorithm no modelo RAM é representado como uma sequência de operações básicas
- Exemplo:

1 operação

```
SomaVetor(V,n)//V: vetor, n: tamanho de V
    soma = 0
    Para i=0 até n-1 faça
        Para j=i até 2i faça
        soma = soma + V[ i ]
    retorna soma
```

executada n vezes

executada i+1 vezes

1 operação

1 operação

$$T(n) = 1 + 1\sum_{i=0}^{n-1}(i-1) + 1 =$$

$$= 2 + \sum_{i=1}^{n} i$$

$$= 2 + \frac{n(n+1)}{2} = \frac{n^2+n+4}{2}$$

RAM Model

- Resumindo:
  - Um modelo de máquina qualquer ainda permite considerar várias outras opções na análise para um algoritmo.
  - Todavia, as análises baseadas no modelo RAM permitem previsões relevantes do desempenho de algoritmos quando implementados em máquinas reais.